

Chapter 1: Designing a Large-Scale Social Feed Ranking System

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The explanations, trade-off analysis and diagrams are my own. Notes marked **Beyond the book** are material I added.

The design in one paragraph. Retrieve a few thousand candidate posts from several cheap sources (ANN over two-tower embeddings, the social graph, fresh posts from followed authors). Merge and deduplicate them and attach features. Score them with a **shared-bottom multi-task network** that predicts several behaviors at once: like, comment, share, long dwell, hide, block. Combine those calibrated probabilities into one utility score with product-chosen weights, rank, apply policy rules, and roll out through shadow → canary → A/B.

Ranking-system design map

The chapter moves from product behavior to retrieval, ranking, and safe rollout.



Working assumption

500M DAU and ~200 ms ranking latency are design assumptions, not claims about a specific platform.

Figure 1: Ranking-system design map

1. Scope and requirements

- **Input:** a user opening their feed (user ID plus request context).
- **Model output:** probabilities of several engagement behaviors for each candidate post.
- **Service action:** an ordered list of posts.
- **Goal:** more *meaningful* engagement and a better user experience, not raw clicks.
- **Out of scope:** ads (Chapter 2), content moderation (Chapter 4; assume unsafe posts are already filtered).

Working assumptions (for scale reasoning, not claims about a real platform):

Quantity	Value	Consequence
daily active users	~500M	~58K feed requests/s on average at 10 requests/user/day; plan for 2-3× at peak
latency budget	~200 ms end to end	only a few thousand candidates can be scored by the heavy model
availability	high; degrade gracefully	fall back to a cached or heuristic feed if ranking times out

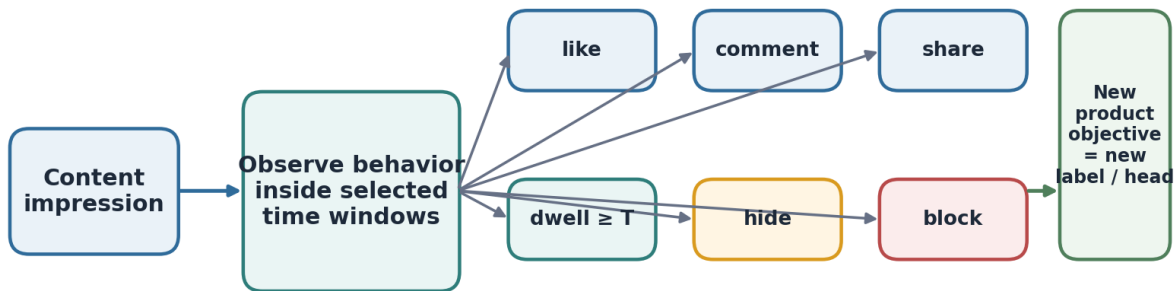
A latency budget I'd propose for the 200 ms:

candidate retrieval (parallel sources)	~30 ms
feature hydration	~40 ms
multi-task ranking (~1-2K candidates)	~80 ms
policy, diversity, dedup	~20 ms
network and slack	~30 ms

2. Labels: one impression, many objectives

From one impression to multiple product objectives

An extensible label system lets one event support several behaviors and future task heads.



Each head gets a binary label: 1 = the behavior happened inside its window, 0 = it did not.

Figure 2: Product objectives and labels

Each impression of a post can produce several binary labels, each defined inside a time window after the impression:

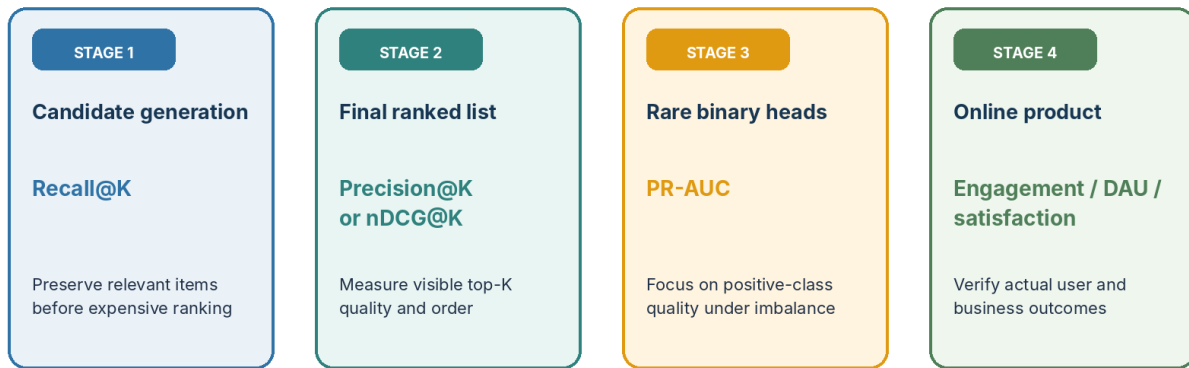
Label	Positive when...	Note
like / comment / share	the action happens within, say, 24 h	comments and shares are rarer and signal stronger interest
dwell $\geq T$	the post stays on screen $\geq T$ seconds	catches passive interest; T is a product choice (e.g. the 70th-percentile dwell)
hide / block	the user hides the post or blocks the author	negative signals, rare but important

Why several labels instead of one “engaged” label? Different behaviors mean different things, and the product wants to weight them differently. A new objective (e.g. “saved”) just becomes a new label and a new head. Labels arrive with **delay**: a share can happen hours later, so training data for day d is only final after the label window closes.

3. Metrics by stage

Choose metrics by stage and product question

No single offline metric represents retrieval quality, final ranking, rare behaviors, and online impact.



Rule of thumb: use a metric only when it matches the stage, ranking depth, label definition, and product decision.

Figure 3: Metrics by stage

Stage	Metric	Question
candidate generation	Recall@K: fraction of the posts the user engaged with that appear in the K candidates	did retrieval keep the good posts?
final ranking	nDCG@K, Precision@K, MRR, mAP	is the visible top of the list well ordered?

Stage	Metric	Question
rare heads (hide, block, share)	PR-AUC, calibration	does each head find its rare positives, and are its probabilities accurate?
overall discrimination	ROC-AUC	does the score separate positives from negatives?
online	meaningful interactions per DAU, time well spent, retention; guardrails: hide rate, reports, latency	does the product improve?

For graded relevance (e.g. share > comment > like > none), nDCG rewards putting the best items first:

$$\text{DCG@}K = \sum_{i=1}^K \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad \text{nDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K} \in [0, 1].$$

Recall@K can look small and still be fine. If a user engaged with 400 posts last month and $K = 50$, recall is at most 12.5%. That’s a property of the definition, not a bad model. Compare recall between models at the same K , or use a per-session version.

4. Model alternatives

Model	Role	Strength	Weakness
collaborative filtering (matrix factorization)	baseline retrieval	learns community taste from interactions alone	cold start for new users and posts
content-based	cold-start retrieval	works for brand-new posts from text, image and topic	misses “people like you also liked”
two-tower	production retrieval	user and item towers embed independently, so item embeddings are precomputed and searched with ANN	only a dot product links user and item, so no rich cross-features
shared-bottom multi-task DNN	final ranker	full user $\times$ post $\times$ context interactions, one head per behavior	too expensive to run on the whole corpus

Two-tower as a generalization of MF. With only ID embeddings in each tower, a two-tower model is matrix factorization. Adding text, metadata, context and history features makes it a hybrid of collaborative and content-based retrieval. Train with in-batch negatives (softmax over the other items in the batch) and correct for the fact that popular items appear more often as negatives, by subtracting $\log q_j$ (the item’s sampling probability) from its logit (Yi et al., 2019).

Shared bottom vs mixture of experts. In a shared-bottom model all heads read one shared representation. That’s efficient and helps rare tasks borrow signal from common ones, but tasks that conflict (like vs hide) can pull the shared layers in opposite directions (negative transfer). **MMoE** (Ma et al., 2018) replaces the shared bottom with several expert networks and a per-task softmax gate that mixes them, so each task gets its own blend. I’d start with shared-bottom and switch to MMoE if per-task validation shows one head degrading when others are added.

Model alternatives: baseline, richer retrieval, and final ranking

The alternatives solve different stages and can be combined rather than treated as one winner.

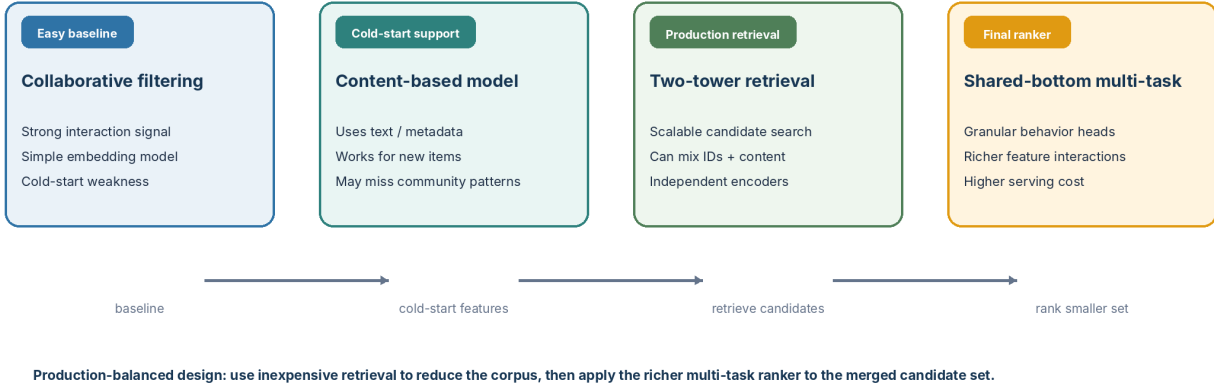


Figure 4: Model alternatives

5. Architecture and the serving score

Inputs: user features, post features, history and context, and **user-author relationship features** (friend or follower, closeness, interaction frequency, mutual friends). Relationship signals are *inputs*, not a separate output.

Two kinds of weights, kept separate:

- **training weights** $\lambda_i \geq 0$ balance the task losses: $L = \sum_i \lambda_i L_i$ (binary cross-entropy per head). They're an optimization tool;
- **serving weights** w_i express product value and can be **negative**:

$$\text{score}(u, p) = \sum_i w_i \hat{P}_i(u, p), \quad \text{e.g. } w = \{\text{like} : 1, \text{comment} : 4, \text{share} : 6, \text{hide} : -20, \text{block} : -50\}.$$

Example. Post A has $P(\text{like}, \text{comment}, \text{share}, \text{hide}) = (0.10, 0.02, 0.01, 0.005)$, so its score is $0.10 + 0.08 + 0.06 - 0.10 = 0.14$. Post B is $(0.20, 0.01, 0.005, 0.03)$: $0.20 + 0.04 + 0.03 - 0.60 = -0.33$. B gets twice the likes but ranks far below A, because of its hide risk.

Warning. > **The sum only means something if each $\hat{P}_i$ is calibrated.**

Heads trained on downsampled data, or with different loss weights, can be systematically over- or under-confident. Check reliability curves per head, and recalibrate (Platt or isotonic scaling on a recent holdout) before combining. Otherwise changing λ_i silently changes the ranking.

6. Data, features and freshness

Family	Examples	Freshness
user	ID embedding, language, location, long-term topic interests	daily batch

Family	Examples	Freshness
post	text/image embeddings, topic, author, age of post	computed once at upload
interaction history	last N posts engaged, per-topic counts in 1 h / 1 d / 7 d windows	near-real-time (streaming)
relationship	friend/follow, closeness, likes on this author in 30 d	hourly-daily
engagement so far	post's likes/comments in the last hour (velocity)	real-time
context	time of day, device, network, session depth	request time

An **aggregated** feature summarizes many events (7-day like count). A **delayed** feature is computed asynchronously and may lag. One feature can be both. Every feature must be logged **as it was at serving time** (point-in-time correctness). Training on today's value of a feature for yesterday's impression is leakage.

7. Training

- **Examples:** one row per (user, post, impression time) with all heads' labels. Non-impressed posts aren't negatives.
- **Loss:** $L = \sum_i \lambda_i L_i$. Choose λ_i by per-head validation metrics and the final ranking metric. Principled options include uncertainty weighting (Kendall et al., 2018) and GradNorm (Chen et al., 2018), which equalizes the gradient magnitudes of different tasks.
- **Imbalance:** positives for share and block are rare. Downsample negatives for efficiency, then correct the probabilities (see Chapter 2, §3).
- **Position bias:** users click what's on top, whatever its quality. Add position as a training feature through a small separate tower, and set it to a fixed value at serving time.

8. Candidate generation, merging and ranking

Several sources run **in parallel**, each with a budget:

Source	Finds	Budget (example)
ANN over two-tower embeddings	posts similar to the user's interests	500
social graph	recent posts from friends and followed accounts	800
trending / fresh	new posts with high early engagement	200
other specialized sources	groups, topics followed	200

Merge, **deduplicate by post ID**, and keep a feature for *which sources* found each post. A post found by both ANN and the graph is a stronger candidate, so that agreement shouldn't be thrown away. Then hydrate features and rank. Tune the source budgets offline by recall@K per source, and online by the share of final impressions each source contributes.

9. The indexing service

The indexing service builds and refreshes the searchable structures that retrieval queries. **Vector quantization** / **product quantization** are compression techniques *inside* it, not separate

Shared-bottom multi-task ranking architecture

Training-loss weights and serving-time utility weights solve different problems.

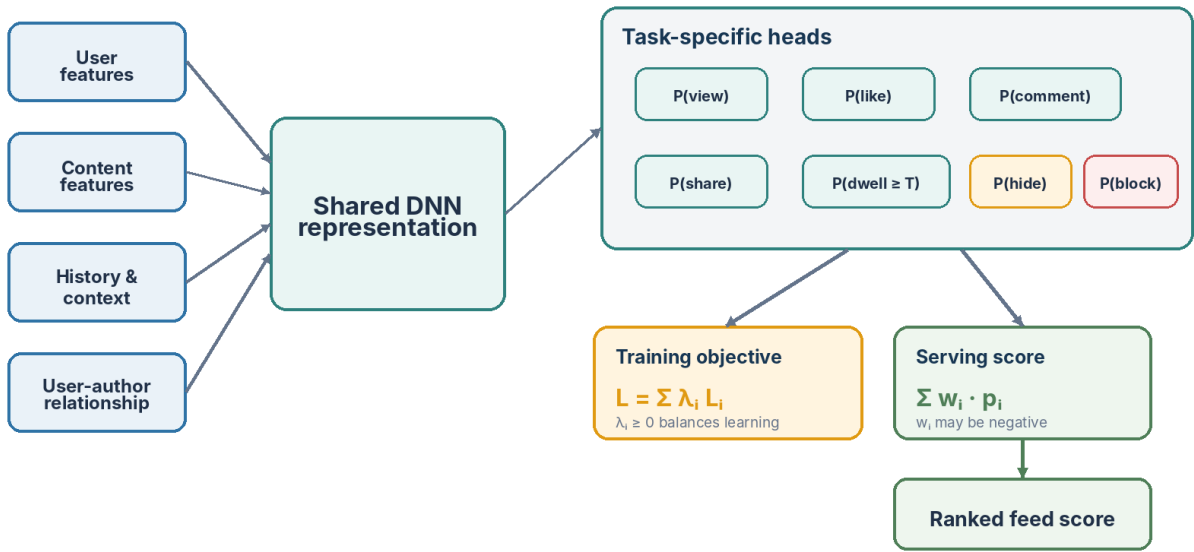
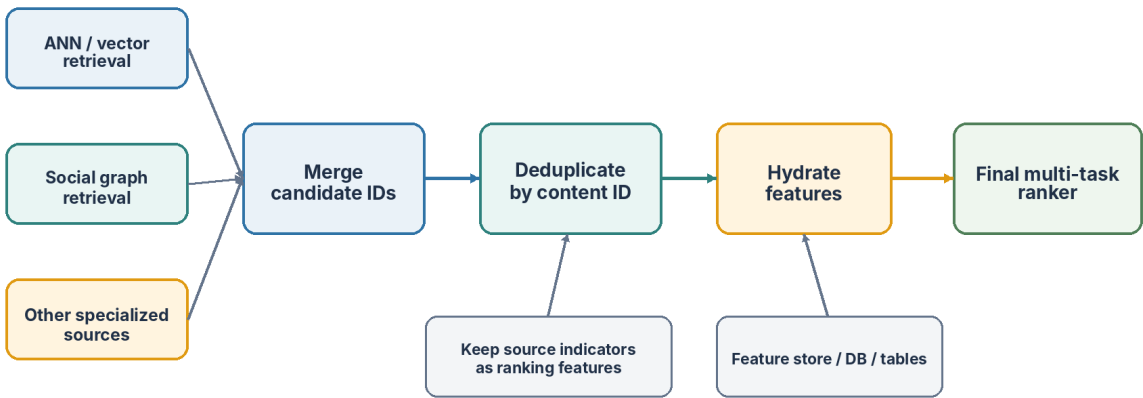


Figure 5: Multi-task architecture

Candidate generation, merge, hydration, and final ranking

Multiple retrieval sources return IDs; the ranking service scores one deduplicated, enriched set.

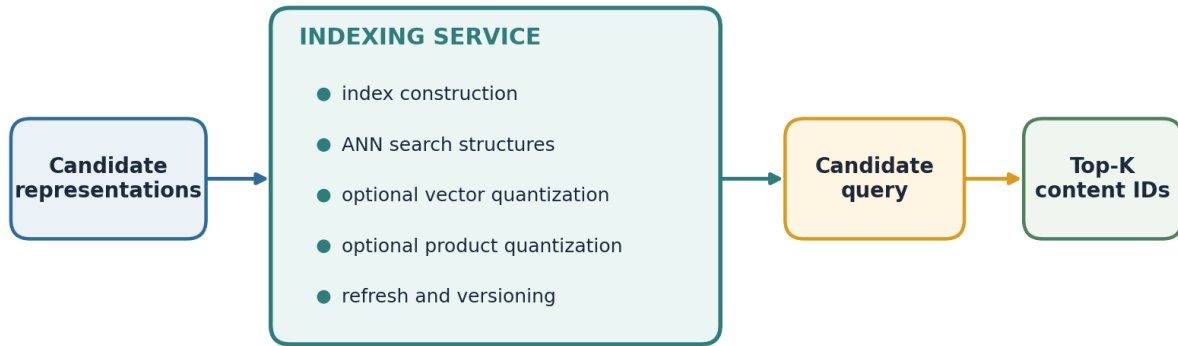


A duplicate retrieval hit is not wasted information: agreement across sources can become a useful ranking feature.

Figure 6: Candidate generation and serving

Indexing service as a retrieval abstraction

Vector quantization and product quantization are optional techniques inside the retrieval service boundary.



Service boundaries

Feature store = reusable features | Indexing service = searchable representations | Ranker = enriched candidate scoring

Figure 7: Indexing service

objectives.

Why compression matters: $1\text{B posts} \times 128\text{-dim float32 embeddings} = 10^9 \times 128 \times 4\text{ B} \approx 512\text{ GB}$. Product quantization with 16 one-byte codes per vector brings that to $\approx 16\text{ GB}$, small enough to fit in memory on a few machines, at a small cost in recall. HNSW graphs give high recall at low latency. IVF-PQ trades some recall for much less memory.

Service boundaries:

- **feature store:** reusable features for training and serving, with point-in-time snapshots;
- **indexing service:** embeddings $\rightarrow$ ANN index, refresh and versioning;
- **candidate-generation service:** queries the index and other sources;
- **ranking service:** hydrates features and scores the merged candidates.

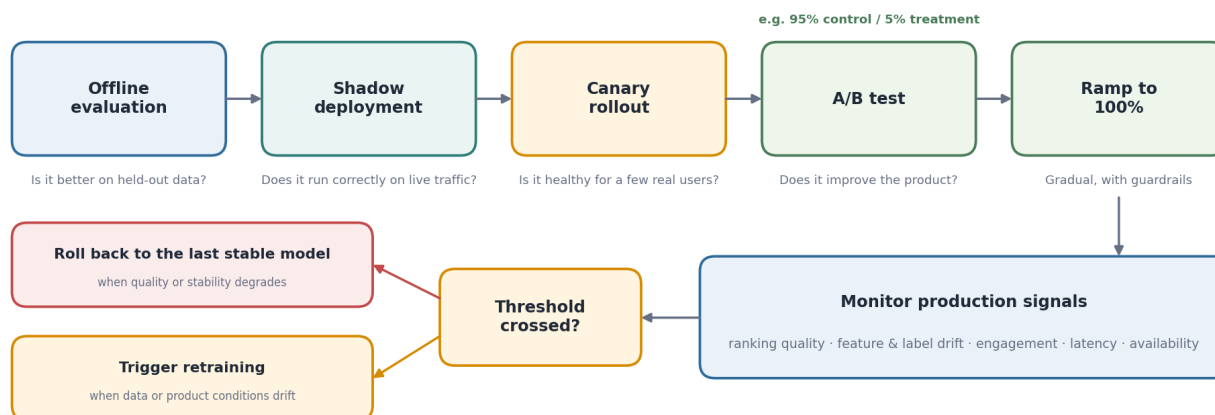
10. Validation and rollout

1. **Offline gate:** better nDCG and per-head PR-AUC on a time-based holdout, with calibration within tolerance.
2. **Shadow:** score live traffic without serving it; check latency, errors and score distributions.
3. **Canary:** serve a small slice of traffic; watch crash, timeout and hide rates.
4. **A/B test:** e.g. 5% treatment vs 95% control for about 2 weeks (to cover weekly cycles and novelty effects); primary metric plus guardrails.
5. **Ramp** 5% $\rightarrow$ 25% $\rightarrow$ 50% $\rightarrow$ 100%, with rollback available at every step.

An A/B test asks *does it make the product better?* A canary asks *is it safe to run?* They answer different questions, which is why both are in the sequence.

Validation, safe rollout, and monitoring

Each stage answers a different question; rollback stays available at every step.



Retraining options: fine-tune with regularization · add adapters / new layers · replay a sample of older data to limit forgetting

Figure 8: Validation and monitoring

11. Monitoring and continual learning

Monitor ranking quality (online nDCG proxies, CTR by position), per-head calibration, feature drift (population stability index on key features), label drift (base rates), engagement, latency and availability.

Retraining can be scheduled (e.g. daily fine-tuning on the latest window) or **triggered** (calibration error or drift beyond a threshold). Options for updating without forgetting:

- **regularization** toward the previous weights (e.g. an L2 penalty on the change, or EWC);
- **adapters or new layers** trained on new data while the core is frozen;
- **replay** of a sample of older data. This reduces forgetting but shifts the training distribution toward the past, so keep the replay ratio small and time-aware.

12. Reference architecture

Offline / async: interaction logs → label generation → feature pipelines → feature store → training → model registry; post embeddings → ANN index. **Online:** request → parallel candidate sources → merge + dedup + hydrate → multi-task ranker → utility score → policy and diversity → feed. Impressions and interactions are logged back for evaluation and training.

13. Decisions and when I'd change them

Area	Starting choice	Switch when (measured)
objective	multi-behavior heads + utility score	a single "meaningful interaction" head matches online results in an A/B test
retrieval	two-tower ANN + graph + fresh sources	one source alone reaches $\geq 95\%$ of the multi-source recall@K at lower latency
ranker	shared-bottom multi-task	a head loses $> 1\%$ relative PR-AUC when trained jointly (negative transfer) → MMoE

Area	Starting choice	Switch when (measured)
serving weights w_i	set by product, tuned by A/B	an experiment on weight vectors finds a better trade-off on the primary metric vs hide rate
index	HNSW or IVF-PQ	exact search fits the latency budget (small corpus)
rollout	offline $\rightarrow$ shadow $\rightarrow$ canary $\rightarrow$ A/B $\rightarrow$ ramp	a low-risk change (e.g. a weight tweak) can skip shadow
retraining	daily fine-tune + drift triggers	calibration error stays within tolerance for weeks $\rightarrow$ weekly

14. Questions and answers

Q: How should the multi-task loss weights λ_i be chosen?

Start equal. Adjust so no head's validation metric is worse than when trained alone, and use uncertainty weighting or GradNorm if the heads' loss scales differ a lot. Judge by the final utility ranking (nDCG of the combined score), not by any single head.

Q: When would MMoE beat shared-bottom?

When tasks are weakly related or conflicting (engagement vs hide/block) and there's enough data per task. Symptom: adding a task hurts another task's metric. With few, closely related tasks, shared-bottom is simpler and just as good.

Q: Which features need real-time freshness?

Session context, the user's last few interactions, and post engagement velocity. These change within minutes and carry most of the short-term signal. Long-term interests and embeddings can be batch-updated daily.

Q: How should candidate-source budgets be set?

Offline, by each source's marginal recall: how many engaged posts it finds that no other source finds. Online, by the fraction of final impressions and engagements each source contributes. Shrink sources whose candidates rarely survive ranking.

Q: Which offline results should gate an online test?

No regression on any head's PR-AUC, an improvement in the combined score's nDCG@K on a time-based holdout, calibration error below a set tolerance, and inference latency within budget.

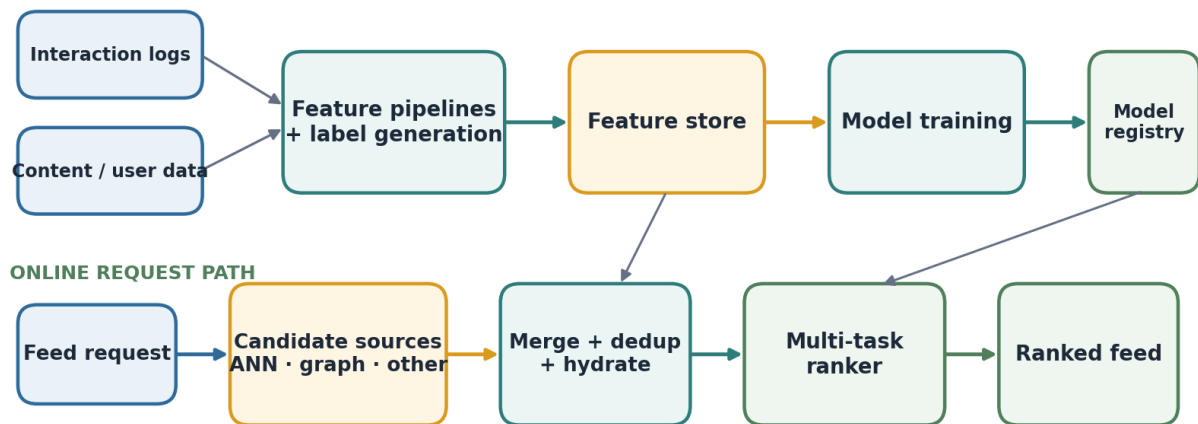
Q: What should trigger retraining?

Calibration error (e.g. expected calibration error) or base-rate drift beyond a threshold, a drop in online engagement relative to control, or a population stability index above about 0.2 on important features.

One defensible large-scale ranking architecture

A hybrid retrieval-and-ranking system under the current assumptions.

OFFLINE / ASYNCHRONOUS



Feedback loop: impressions and interactions return to logs, enabling evaluation, monitoring, and future updates.

Figure 9: Reference architecture